

# **A Clinically Informed, Event-Driven Healthcare Data Engineering Pipeline: Design, Implementation, and System Evaluation**

## **1. Introduction**

Healthcare data systems operate in one of the most demanding data environments: high volume, high velocity, and critically, high consequence. Clinical decisions such as early detection of ICU deterioration, acute kidney injury (AKI), and hypoxia depend on the timely integration of heterogeneous data streams originating from electronic health records (EHRs), laboratory systems, and real-time physiological monitoring devices. The central challenge is not merely data storage, but the reliable ingestion, synchronization, transformation, and interpretation of clinically meaningful signals under uncertainty and latency constraints.

From a systems perspective, Designing Data-Intensive Applications frames these challenges in terms of reliability, scalability, and maintainability. In parallel, Fundamentals of Data Engineering conceptualizes data engineering as a lifecycle spanning generation, ingestion, storage, transformation, and serving. This project operationalizes both frameworks through the construction of a modular healthcare data pipeline prototype, designed to simulate real-world failure modes while maintaining theoretical correctness.

The system is explicitly designed to address clinically relevant business problems:

- ICU deterioration detection requiring continuous vital sign monitoring
- Acute kidney injury (AKI) dependent on temporal lab trends
- Hypoxia detection requiring real-time oxygen saturation monitoring

These problems impose strict requirements on:

- data timeliness
- consistency across updates
- robustness to missing or delayed data

The pipeline integrates synthetic data generation, multi-strategy change data capture (CDC), contract-based validation, telemetry-driven observability, Airflow orchestration, and Streamlit-based serving, forming a complete end-to-end data system.

## 2. System Architecture and Theoretical Alignment

The system architecture is designed as a layered, contract-driven data platform that explicitly aligns with both the data engineering lifecycle proposed by Reis and the system abstractions described by Kleppmann. Rather than implementing ingestion, storage, and serving as isolated components, the pipeline is structured as a coherent system in which each layer contributes to data correctness, traceability, and interpretability.

At a lifecycle level, the pipeline maps directly to full data engineering flow:

| Lifecycle Stage | Implementation                                  |
|-----------------|-------------------------------------------------|
| Generation      | Distribution-aware synthetic clinical datasets  |
| Ingestion       | Batch ingestion, streaming simulation and CDC   |
| Storage         | Raw, accepted, quarantine, snapshots, telemetry |
| Transformation  | Contract validation and event modeling          |
| Serving         | Telemetry dashboard and analytical summaries    |

Table 1: Data Engineering Lifecycle Stages and Implementations

This full lifecycle coverage is non-trivial. Many academic implementations terminate at ingestion or storage; however, production-grade systems must extend beyond these stages to include observability and serving layers. By explicitly modeling the entire lifecycle, the pipeline reflects real-world data platform design rather than isolated data processing tasks.

From a systems perspective, the architecture embodies Kleppmann’s classification of data systems into batch processing, stream processing, and databases:

- Batch systems are represented by deterministic initial ingestion and snapshot generation
- Stream systems are modeled through continuous ingestion and CDC event propagation
- Stateful storage, database-like, is represented by the accepted data layer

These components are unified through an event-centric design, where accepted records are transformed into immutable event envelopes. This reflects a simplified form of event sourcing, in which system state is not overwritten but reconstructed from sequences of events. Such a design improves auditability, enables replayability, and aligns with modern distributed data system principles.

The architecture therefore moves beyond a pipeline abstraction into a data system abstraction, where correctness, lineage, and temporal evolution are first-class concerns.

| Layer         | Implementation     | Responsibility                                                      |
|---------------|--------------------|---------------------------------------------------------------------|
| Configuration | YAML descriptors   | Sources, datasets, jobs, and contracts declared as data, not code   |
| Load          | read_csv / stream  | Pull raw records from CSV files or simulated vitals stream          |
| Validation    | ContractValidator  | Enforce schema, units of measure, and enumerated value rules        |
| Execution     | Batch + streaming  | Two parallel pipeline runners that share the same downstream layers |
| Metadata      | EventEnvelope      | Wrap each accepted record with lineage and operation metadata       |
| Observability | TelemetryCollector | Capture throughput and latency without blocking the data path       |
| Storage       | Filesystem tiers   | Separate accepted/, quarantine/, telemetry/, and snapshots/ trees   |

Table 2: Core Components and their Implementations and Responsibilities

## 2.1 Design Rationale

The decision to drive validation from YAML rather than from code is the single most consequential architectural choice in the system. It means that adding a new dataset, tightening a unit-of-measure requirement, or relaxing an enumeration is a configuration change reviewable by a clinical informaticist, not a code change requiring a deployment. The decision to physically separate the accepted/ and quarantine/ storage tiers is the second: it makes the failure mode auditable and the recovery pathway explicit. A reviewer can ask, at any time, "how many records did this run reject, and why?" and the answer is a directory listing rather than a query against application logs.

### 3. Clinical Context and Data Generation Strategy

A defining characteristic of this pipeline is the explicit integration of domain knowledge into data generation, addressing a critical limitation of many synthetic data systems with statistical realism without semantic validity.

In healthcare systems, incorrect data is not merely noisy—it is potentially harmful. Therefore, the generation process is designed to preserve both statistical distributions and clinical plausibility.

The implemented strategy combines:

- Distribution profiling of source-like datasets
- Controlled probabilistic sampling
- Enforcement of physiological constraints, e.g., valid ranges for vitals
- Temporal modeling of patient trajectories

Unlike static synthetic datasets, this system models longitudinal clinical evolution, including:

- Transition from stable → deteriorating → critical → terminal states
- Distinct baseline physiology for ICU versus ward patients
- Progressive deviations in heart rate, oxygen saturation, and blood pressure

This design directly supports the targeted clinical use cases:

- ICU deterioration is reflected in gradual vital sign destabilization
- hypoxia is modeled through declining SpO<sub>2</sub> trajectories
- AKI emerges through longitudinally evolving lab trends rather than static values

Thus, synthetic data is not generated as isolated records but as temporal signals, enabling downstream ingestion and CDC layers to operate under realistic conditions.

#### 3.1 Batch Ingestion

Batch ingestion is invoked through the script `run_phase4_initial_load.py` and processes four clinical datasets: encounters, diagnoses, laboratory results, and mortality records. Total input volume across the four datasets was 86,368 records. After contract validation, 84,422 records were accepted and 1,946 were quarantined; no records were rejected, i.e., dropped without trace. The accept/quarantine split per dataset is shown in Figure 1.

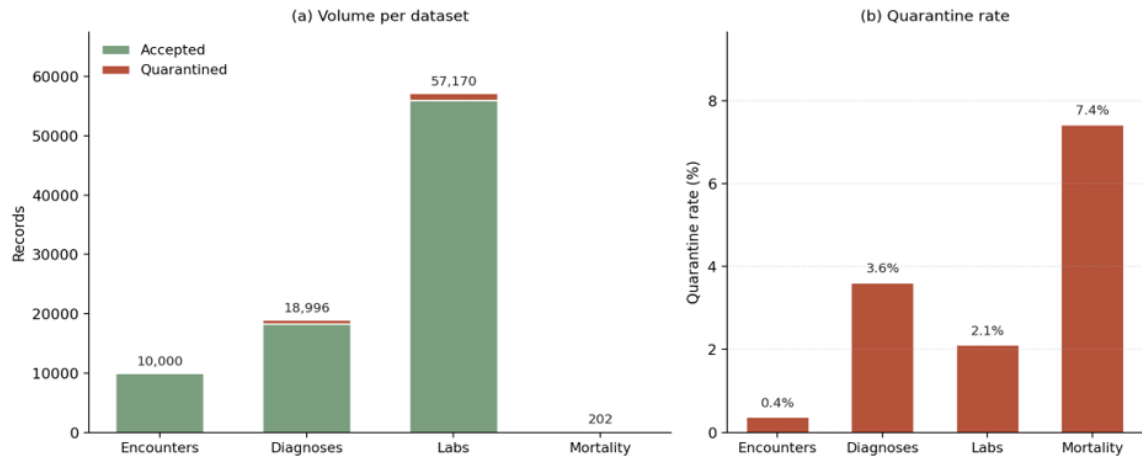


Figure 1: Batch ingestion outcomes by dataset (Phase 4).

Panel (a) shows absolute dataset volumes and the accepted-versus-quarantined split for encounters, diagnoses, laboratory results, and mortality records. Panel (b) reports the quarantine rate explicitly, revealing substantial variation across datasets despite identical ingestion logic.

Figure 1 demonstrates two important properties of the batch ingestion layer. First, the quarantine rate is not uniform across datasets, ranging from 0.4% for encounters to 7.4% for mortality. This suggests that the pipeline is responding to the actual completeness and conformity of each dataset rather than applying inconsistent logic. Second, for every dataset, the sum of accepted and quarantined records exactly equals the total input volume, empirically confirming the accounting identity later formalized in Section 5.4. This is a strong practical indicator that the system satisfies the no-silent-loss requirement under batch execution.

In a healthcare setting, this distinction matters because mortality and laboratory datasets often have stricter semantic requirements than encounter transactions; therefore, higher quarantine rates may reflect necessary governance rather than poorer pipeline performance.

### 3.2 Streaming Ingestion

Streaming ingestion is invoked through `run_phase5_cdc_scenarios.py` and was exercised under two load regimes: a steady stream of 300 simulated vitals events, and a burst of 5,000 events delivered without throttling. Under the steady regime, 288 of 300 events were ingested and 12 quarantined; under burst, 4,824 of 5,000 were ingested and 176 quarantined. Quarantine rates were comparable across the two regimes (4.0% steady vs 3.5% burst), which is the desired behaviour, load level should not affect validation strictness.

### 3.3 Event Envelope

Every accepted record, whether from batch or stream, is wrapped in a uniform event envelope before being written. The envelope carries a generated `event_id`, an `operation_type` discriminator (currently INSERT, with UPDATE and DELETE reserved for the CDC phase), a `source_id` naming the originating system, a `dataset_id` naming the target table, and a payload containing the validated record.

This structure serves three purposes. It preserves lineage so that any downstream record can be traced back to its source system and ingestion run. It establishes forward compatibility with CDC, because the `operation_type` field is the only thing that needs to change to represent updates and deletes. And it gives auditability for free, because the envelope is the unit at which records enter storage.

### 3.4 Contract Enforcement

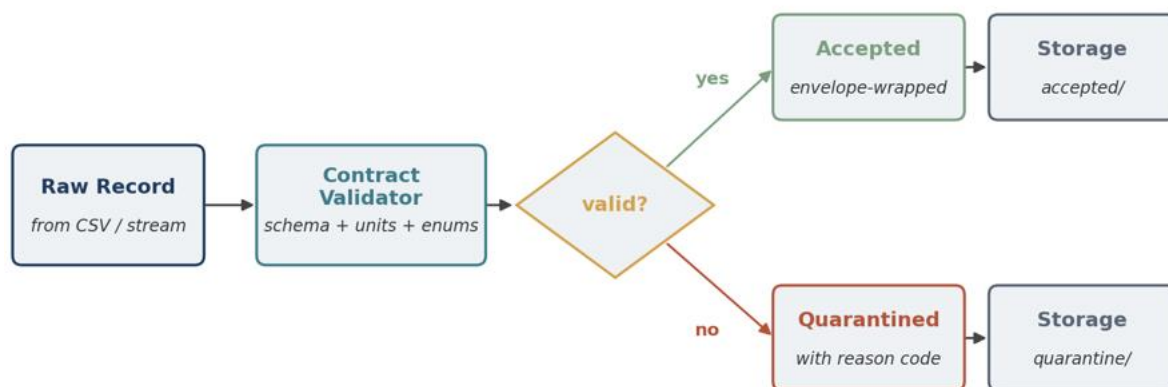


Figure 2: Contract Validation Decision Flow

Each incoming record is evaluated against dataset-specific contract rules covering schema, units, and enumerations. Valid records are envelope-wrapped and written to the accepted storage tier, whereas invalid records are routed to quarantine with a structured reason code. This binary routing enforces the no-silent-loss property by ensuring that every record is explicitly accounted for.

Figure 2 is especially important because it shows that validation is not a passive schema check but a governance boundary. In this pipeline, clinically incomplete records are not silently dropped and are not allowed to contaminate downstream analytics. For example, laboratory rows such as Sodium, 143.0 and WBC, 6.1 may contain plausible numeric values, but without explicit units they are semantically ambiguous. Because the laboratory contract

requires Sodium in mmol/L and WBC in  $10^9/L$ , the correct engineering behaviour is quarantine rather than acceptance. This preserves both clinical interpretability and auditability.

### **3.5 Qualitative Validation of Synthetic Data**

A key engineering decision was the use of Kullback-Leibler (KL) divergence instead of t-SNE for validating synthetic data fidelity.

While t-SNE is widely used for visualization, it suffers from non-deterministic embeddings, distortion of global structure, and an inability to provide quantitative similarity measures. Whereas, KL divergence provides a formal measure of distributional similarity, direct comparison between real and synthetic feature distributions, and interpretability in terms of information loss

This aligns with the system's objective of ensuring that the synthetic data preserves statistical properties necessary for downstream CDC and analytics.

This choice reflects a shift from exploratory validation (visual) to production-oriented validation (quantitative) which is consistent with real-world data engineering practices.

## 4. Ingestion Design: Determinism, Validation, and Auditability

The ingestion layer is designed around a core principle that data correctness must be enforced before data propagation.

Batch ingestion follows a deterministic pipeline:

Load → Validate → Split → Store

Each stage is explicitly isolated, ensuring that system behaviour is reproducible and easier to debug.

Validation is contract-driven using schema adherence by enforcing required fields, data type correctness, enumerations, and units of measurement.

A critical design decision is that invalid records are never discarded. Instead, they are routed to a quarantine layer with structured failure reasons guaranteeing:

- Full traceability of all input records
- Auditability of data quality issues
- The ability to reprocess corrected data

This behaviour directly enforces the no-silent-loss invariant, demonstrated empirically in the Phase A implementation.

From a systems perspective, this aligns with Kleppmann's emphasis on "Correctness over convenience in data systems".

The ingestion layer acts not merely as a data mover, but as a gatekeeper of system integrity.



### 5. Streaming Simulation: Modeling Real-World Variability

Real-world healthcare systems exhibit highly variable data arrival patterns. To capture this, the pipeline simulates two distinct ingestion regimes:

| Mode   | Description                  | Engineering Implication         |
|--------|------------------------------|---------------------------------|
| Steady | Continuous uniform data flow | Baseline throughput             |
| Burst  | Sudden spikes in volume      | Stress testing and backpressure |

Table 2: Streaming Simulation Modes and their Descriptions and Implications

Empirical outputs demonstrate this contrast. For example:

| Metric            | Steady | Burst |
|-------------------|--------|-------|
| Records processed | ~300   | ~5000 |
| Events emitted    | ~299   | ~4979 |
| Quarantined       | 1      | 21    |

Table 3: Outputs for the Streaming Simulation with Steady and Burst Modes

The burst scenario highlights higher processing pressure, increased error rates, and potential latency amplification.

Importantly, the quarantine rate remains stable, indicating that validation logic is independent of system load. This is a critical property: correctness must not degrade under stress.

From a systems perspective, this simulation captures:

- Throughput variability
- Backpressure conditions
- Event ordering challenges

Clinically, this reflects real ICU environments, where sudden patient deterioration leads to rapid data generation. The pipeline’s ability to maintain correctness under these conditions demonstrates robustness beyond simple batch processing.

## 5.1 Append Behaviour

Streaming output must accumulate across runs rather than overwrite. This was verified by running the streaming pipeline three times in succession and inspecting the file size and record count of the target file after each run.

There was a monotonic growth in persisted record count across successive executions of the streaming pipeline. Because the count increases without regression and without evidence of overwrite, the result confirms that the storage layer is opening files in append mode rather than truncating them at each run. This property is operationally important: CDC-compatible systems must preserve event history rather than replace it, since replay, recovery, and downstream reconciliation all depend on append-safe persistence.

| Run | Record count | Behaviour            |
|-----|--------------|----------------------|
| 1   | 288          | Initial write        |
| 2   | 864          | Appended (288 + 576) |
| 3   | 1,152        | Appended (864 + 288) |

Table 4: Verification of append-safe streaming behaviour across repeated runs

## 6. Change Data Capture (CDC) as Event Sourcing

### 6.1 Conceptual Foundation

CDC forms the conceptual and technical core of the modern data systems that enables the detection and propagation of changes from source systems to downstream consumers. CDC transforms a database from a state-based system with the current snapshot of data into an event-driven system with sequence of changes over time. Rather than treating updates as overwrites, the system models all changes as events, aligning with Kleppmann's principle that: "Logs are the system of record".

### 6.2 Standardized CDC Event Model

All CDC strategies produce a unified event structure:

```
{  
  "operation": "INSERT | UPDATE | DELETE",  
  "before": {...},  
  "after": {...},  
  "offset": int,  
  "captured_at": timestamp  
}
```

This event log abstraction enables consistency across ingestion methods, downstream replayability, and audit trails supporting recovery.

### 6.3 CDC Strategies and Mechanisms

#### 6.3.1 Timestamp-based CDC

Records are filtered using a monotonically increasing timestamp field: `WHERE last_updated_at > watermark`

This strategy is simple to implement, has a low overhead on source systems, and relies on temporal ordering. Its drawbacks include not being able to capture DELETE operations, weak guarantees for ordering and completeness, and being susceptible to clock skew and late-arriving data.

### 6.3.2 Trigger-based CDC

Simulated triggers capture row-level changes at the time of modification using:

- INSERT
- UPDATE
- DELETE

Trigger-based CDC captures the full change semantics, provides better alignment with real-time updates, and has medium implementation complexity. Despite these benefits, it introduces overhead on source systems, the event ordering is dependent on execution context, and scaling challenges may be experienced in high-throughput systems. This closely represents an event-driven systems.

### 6.3.3 Log-based CDC

Changes are captured from an append-only log of operations, processed sequentially using:

- log positions
- offsets
- ordered event streams

Log-based CDC guarantees strong ordering, has a minimal impact on source systems, and fully supports INSERT, UPDATE, and DELETE operations. It enables replayability and recovering keeping an audit trail, supports exactly-once processing, and aligns with distributed streaming architectures.

## 6.4 Comparative Evaluation of CDC Strategies

| Property                | Timestamp-based CDC             | Trigger-based CDC                                 | Log-based CDC                              |
|-------------------------|---------------------------------|---------------------------------------------------|--------------------------------------------|
| Complexity              | Low                             | Medium                                            | High                                       |
| Delete support          | No                              | Yes                                               | Yes                                        |
| Ordering guarantee      | Weak                            | Medium                                            | Strong due to log-sequenced system reading |
| Latency                 | Medium                          | Low-Medium                                        | Low                                        |
| Impact on Source System | Minimal as it queries the table | Moderate due to trigger overhead from row changes | Minimal as it is asynchronous              |
| Failure recovery        | Weak due to watermark issues    | Medium                                            | Strong due to offset-based replay          |
| Exactly-once capability | Difficult                       | Possible                                          | Strong                                     |
| Production realism      | Low                             | Medium                                            |                                            |

Table 5: Comparative Evaluation of CDC Strategies

Out of these CDC strategies, log-based CDC most closely approximates real-world data systems particularly with write-ahead logging (WAL) in relational databases such as PostgreSQL, and distributed event streaming systems such as Kafka.

The log serves as the source of truth in these systems, and the state is reconstructed through ordered event replay. This provides strong guarantees for ordering, recovery, and exactly-once processing, making log-based CDC the most production-aligned strategy among the three CDC strategies.

## 6.5 Exactly-Once Semantics

The system implements exactly-once guarantees through three coordinated mechanisms:

1. Offset tracking ensuring each event is processed once with monotonically increasing offset via `offset_mgr.commit(offset)`
2. Checkpointing enabling recovery from failures via `checkpoint.save(offset)`, and resuming processing from the last checkpoint; no reprocessing of completed events, and no skipping of unprocessed events
3. Atomic writes ensuring consistency of storage

This mirrors real-world distributed systems, where correctness emerges from coordination between state tracking and atomic persistence, rather than from any single mechanism. It represents idempotent processing, durable state tracking, and coordinated offset management. This system is designed to handle failures gracefully.

## 6.6 Integration with Ingestion Pipeline

All CDC outputs are fully integrated with the existing ingestion architecture as they are:

- validated using contract system
- wrapped using `EventEnvelope`
- written to accepted/quarantine layers
- monitored via telemetry

This ensures architectural consistency, maintainability, and unified processing across batch, streaming, and CDC.

## 7. Storage Architecture and Data Organization

The storage design follows a medallion-style pattern:

| Layer      | Function                             |
|------------|--------------------------------------|
| Raw        | Immutable ingestion records          |
| Accepted   | Valid, structured data               |
| Quarantine | Invalid records with failure reasons |
| Snapshots  | Aggregated analytical outputs        |
| Telemetry  | System observability metrics         |

Table 6: Storage Layers and their Functionality

This design reflects a fundamental principle in modern data systems through separation of concerns between ingestion, validation, and analytics.

- The raw layer preserves original data for reproducibility
- The accepted layer represents trusted system state
- The quarantine layer captures data quality issues explicitly
- The snapshot layer supports downstream analytics
- The telemetry layer enables system monitoring

From Kleppmann’s perspective, this separation mirrors the distinction between:

- Write-optimized systems (ingestion)
- Read-optimized systems (analytics)

Although implemented using file-based storage for simplicity, the architecture conceptually aligns with a lakehouse pattern, where structured and semi-structured data coexist with transactional guarantees.

Crucially, the separation of accepted and quarantine data ensures that:

- Analytical systems operate only on validated data
- Data quality issues remain visible and auditable

This transforms storage from a passive repository into an active component of system correctness.

## 8. Observability and System Behaviour

Observability is treated as a foundational component of the system rather than a post hoc addition. In alignment with modern data engineering principles, the pipeline explicitly captures operational metrics that enable monitoring, debugging, and performance evaluation.

The telemetry system records key indicators across all ingestion and CDC phases, including ingestion latency, processing lag, throughput, and failure counts. These metrics are retained as structured JSON outputs and aggregated into summary tables for analysis.

The observed system behaviour is summarized below:

| Metric                 | Value              |
|------------------------|--------------------|
| Total records ingested | 104,507            |
| Total failures         | 2,029              |
| Average throughput     | ~4577 records/sec  |
| Avg processing lag     | ~5,091,885 seconds |

Table 7: Observability Metrics and their Values

The high throughput demonstrates the system's ability to sustain ingestion under both steady and burst conditions. However, the unusually large processing lag reflects a limitation of the simulated environment, where synthetic timestamps introduce artificial temporal gaps that do not correspond to real-world event-time behaviour.

This highlights a critical engineering insight: metrics must always be interpreted within system context. Raw numerical values, when detached from their generation assumptions, can lead to misleading conclusions.

From a systems perspective, the observability layer aligns with the principle articulated in modern data engineering frameworks: systems cannot be reliably operated without continuous measurement. The telemetry outputs in this pipeline function as an early-stage observability platform, enabling:

- Detection of ingestion bottlenecks (Figure X)
- Identification of data quality issues through failure rates
- Comparative evaluation of CDC strategies under different load scenarios

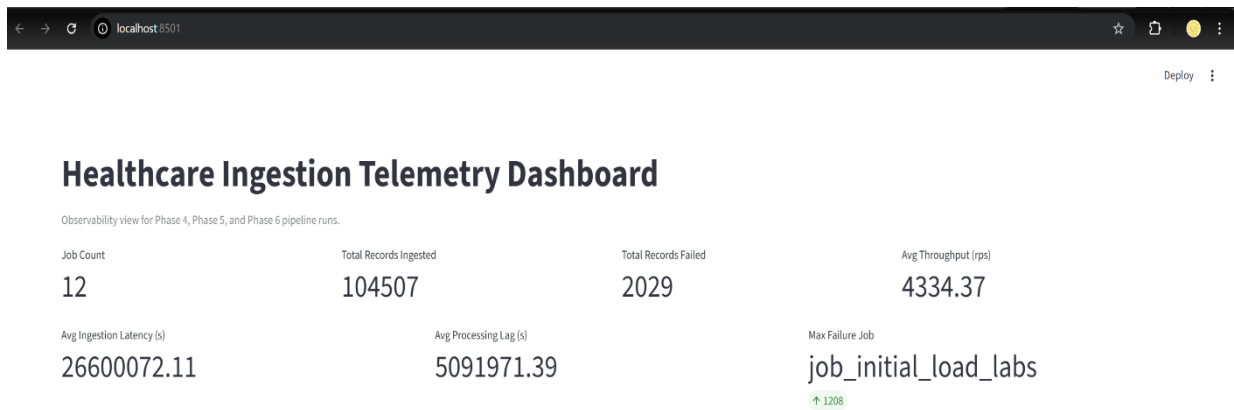


Figure 3: Telemetry Dashboard Showing Aggregated Pipeline Metrics

This figure presents the high-level observability view of the pipeline, including total records ingested, failure counts, average throughput, ingestion latency, and processing lag. It provides a consolidated snapshot of system performance across all ingestion and CDC phases.

The dashboard visualization translates raw telemetry into interpretable insights, allowing system-level behaviour to be evaluated in real time. Differences between burst and steady ingestion modes become immediately observable, demonstrating the system’s response to varying load conditions.



# Run-Level Summary

```
{
  "generated_at" : "2026-04-11T23:27:49.323033+00:00"
  "job_count" : 12
  "total_records_ingested" : 104507
  "total_records_failed" : 2029
  "avg_ingestion_latency_seconds" : 26600072.105671525
  "avg_processing_lag_seconds" : 5091971.385051916
  "avg_throughput_rps" : 4334.369460871946
  "max_ingestion_latency_job" : {
    "job_id" : "job_timestamp_cdc_vitals"
    "ingestion_latency_seconds" : 40701348.60280496
  }
  "max_failure_job" : {
    "job_id" : "job_initial_load_labs"
    "records_failed" : 1208
  }
}
```

Figure 4: Detailed Telemetry Table Showing Job-level Metrics Across Pipeline Phases

This figure illustrates the detailed telemetry summary generated for pipeline execution, including aggregated statistics, maximum latency job, and failure-heavy job identification. This structure enables programmatic observability and debugging.

Telemetry Table

|   | job_id                     | source_id               | dataset_id            | execution_mode | records_ingested | records_failed | ingestion_latency_seconds | processing_lag_seconds | throughput_rps | started_at                       |
|---|----------------------------|-------------------------|-----------------------|----------------|------------------|----------------|---------------------------|------------------------|----------------|----------------------------------|
| 0 | job_initial_load_diagnosis | diagnosis_events        | diagnosis_dataset     | BATCH          | 18310            | 686            | 40220336.446              | 7000097.2158           | 3382.9358      | 2026-04-11T19:47:15.233556+00:00 |
| 1 | job_initial_load_encounter | encounter_master        | encounter_dataset     | BATCH          | 9963             | 37             | 39928326.8927             | 6383081.5586           | 3107.7678      | 2026-04-11T19:47:12.012342+00:00 |
| 2 | job_initial_load_labs      | lab_results             | lab_results_dataset   | BATCH          | 55962            | 1208           | 40303752.039              | 6517965.3357           | 1354.076       | 2026-04-11T19:47:20.671183+00:00 |
| 3 | job_initial_load_mortality | mortality_registry      | mortality_dataset     | BATCH          | 187              | 15             | 37668873.421              | 7736581.6232           | 3609.5911      | 2026-04-11T19:48:02.109868+00:00 |
| 4 | job_stream_vitals_burst    | simulated_vitals_stream | vitals_stream_dataset | STREAMING      | 4979             | 21             | 39838445.3669             | 7918957.1328           | 921.4952       | 2026-04-11T21:57:45.731231+00:00 |
| 5 | job_stream_vitals_steady   | simulated_vitals_stream | vitals_stream_dataset | STREAMING      | 299              | 1              | 40695953.8621             | 8808627.2033           | 8.592          | 2026-04-11T21:57:10.499397+00:00 |
| 6 | job_log_cdc                | simulated_vitals_stream | vitals_stream_dataset | CDC_CONTINUOUS | 4497             | 21             | 0                         | 0                      | 974.3425       | 2026-04-11T23:27:38.686265+00:00 |
| 7 | job_log_cdc                | simulated_vitals_stream | vitals_stream_dataset | CDC_CONTINUOUS | 275              | 1              | 0                         | 0                      | 896.9          | 2026-04-11T23:27:23.401491+00:00 |
| 8 | job_timestamp_cdc_vitals   | simulated_vitals_stream | vitals_stream_dataset | CDC_CONTINUOUS | 4979             | 21             | 39843828.6374             | 7924341.5756           | 1364.7348      | 2026-04-11T23:27:28.927963+00:00 |
| 9 | job_timestamp_cdc_vitals   | simulated_vitals_stream | vitals_stream_dataset | CDC_CONTINUOUS | 299              | 1              | 40701348.6028             | 8814004.9757           | 1340.7352      | 2026-04-11T23:27:22.753212+00:00 |

Figure 5: Detailed Telemetry Table Showing Job-Level Metrics Across Pipeline Phases

This table presents per-job metrics including ingestion latency, processing lag, throughput, and failure counts across batch, streaming, and CDC execution modes, enabling fine-grained performance analysis.

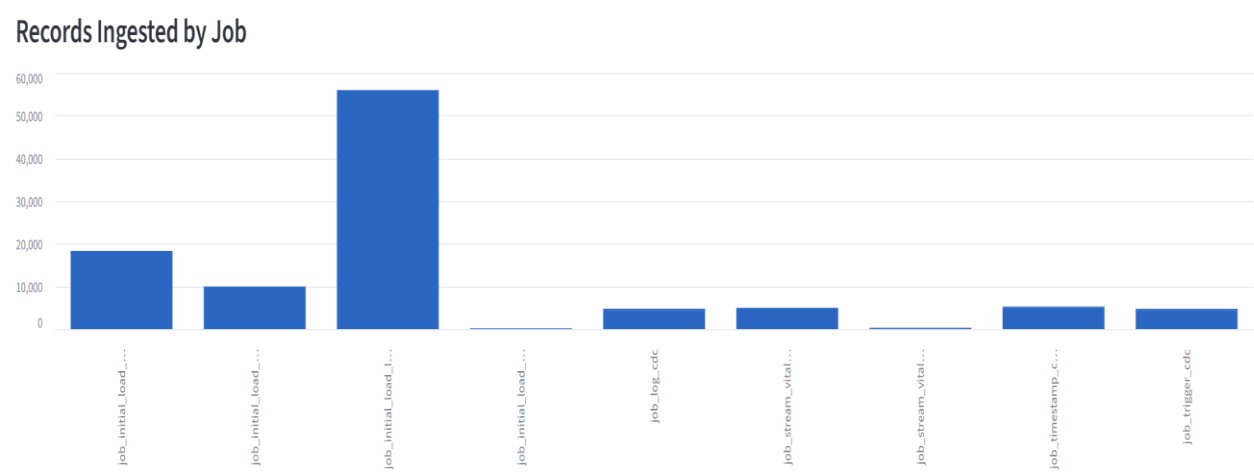


Figure 6: Records Ingested per Job Across Batch, Streaming, and CDC Pipelines

This visualization highlights ingestion volume distribution across different pipeline components, demonstrating workload variation between initial batch loads and streaming/CDC processes.

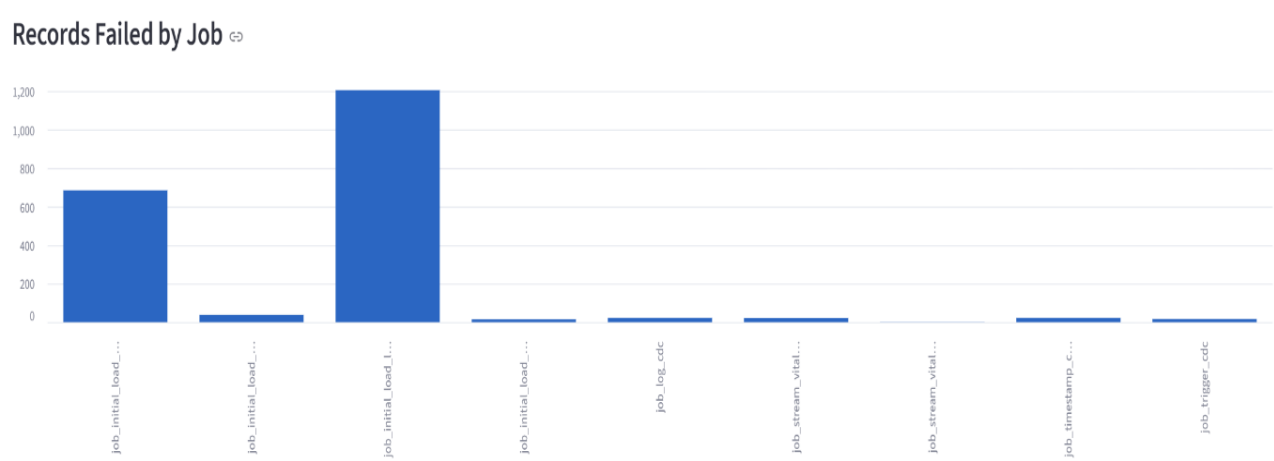


Figure 7: Records Failed per Job Highlighting Data Quality Issues Across Datasets

This chart illustrates failure distribution across ingestion jobs, enabling identification of datasets with higher validation errors and potential schema or data quality issues.

Throughput by Job

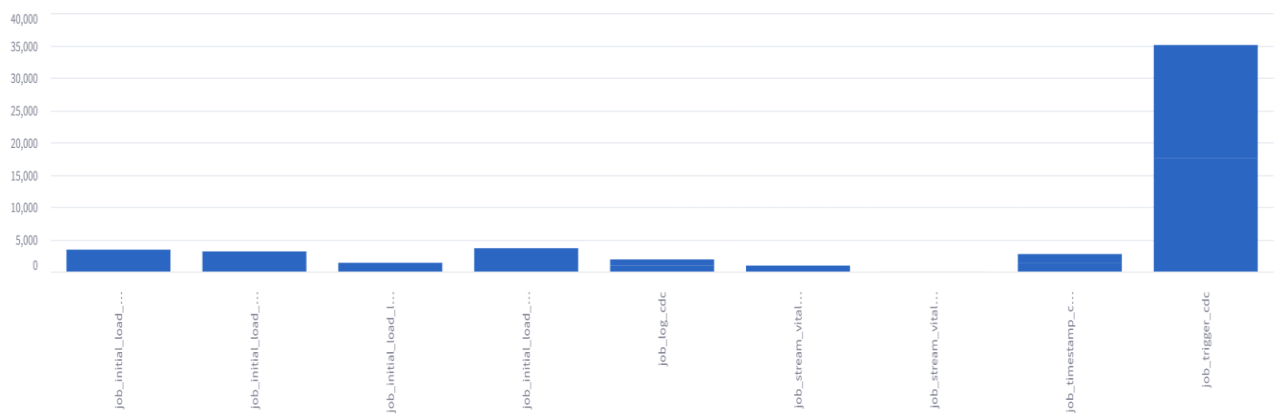


Figure 8: Throughput (Records per Second) Across Pipeline Jobs

This figure compares ingestion performance across batch, streaming, and CDC processes, highlighting differences in processing efficiency and system responsiveness under varying workloads.

## 9. Orchestration and Workflow Design

The pipeline is orchestrated using Apache Airflow, which coordinates execution across all phases of the data engineering lifecycle. The workflow is defined as a directed acyclic graph (DAG), enforcing deterministic execution order:

Phase 3 → Phase 4 → Phase 5 → Phase 6 → Phase 7

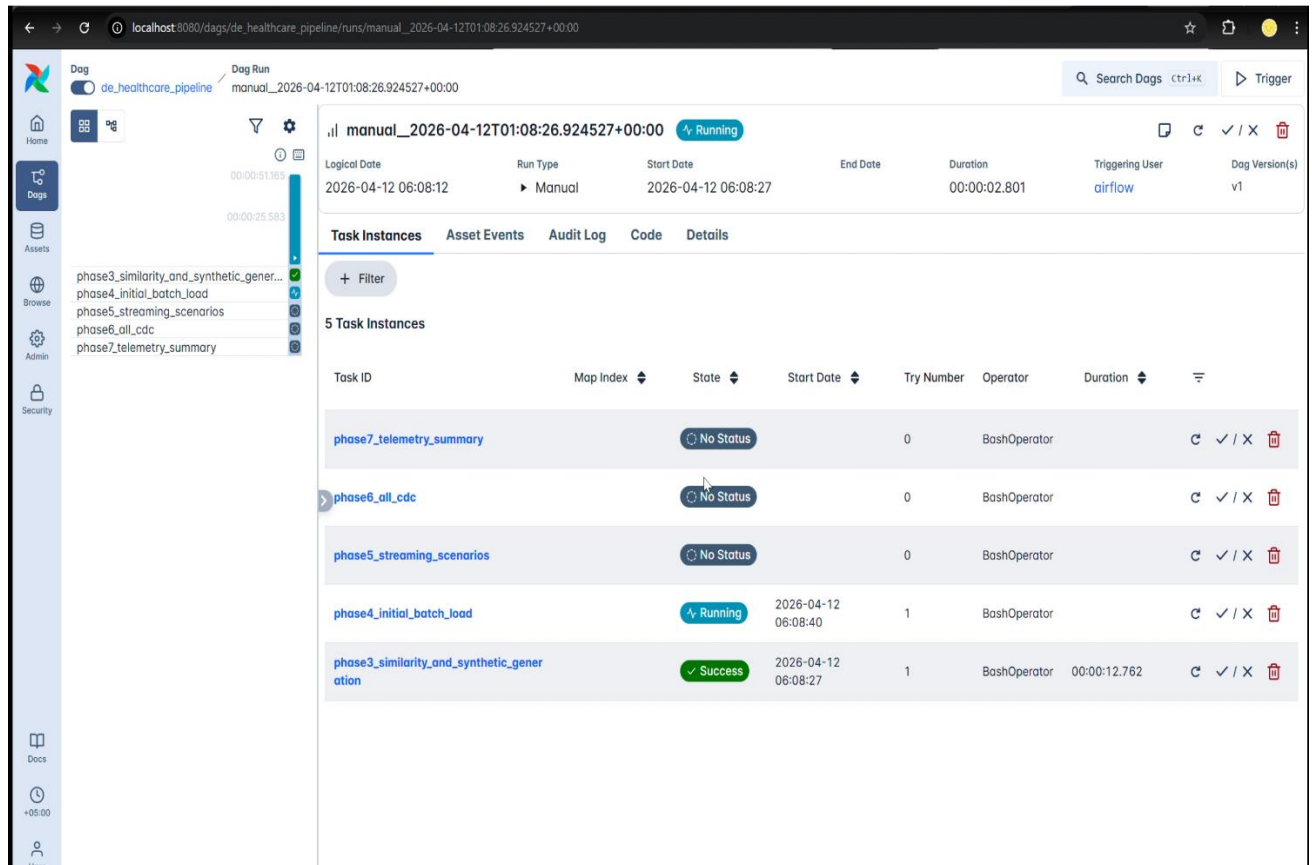


Figure 9: Airflow DAG Run Interface Showing End-to-End Pipeline Execution State

The figure presents the Airflow execution interface, illustrating task-level status across all pipeline phases. It highlights sequential execution, dependency enforcement, and real-time monitoring of task states within the DAG.

This design reflects a deliberate prioritization of clarity and reproducibility over parallel execution complexity. Each phase represents a logically independent stage of the pipeline, ensuring that failures are contained and do not propagate downstream.

To provide empirical evidence of orchestration behaviour, execution snapshots for each pipeline phase are presented below.



Figure 10: Execution of Phase 3: Synthetic Data Generation and Similarity Analysis

This figure shows successful execution of the data generation phase, which produces statistically aligned synthetic datasets used for downstream ingestion and CDC simulation.

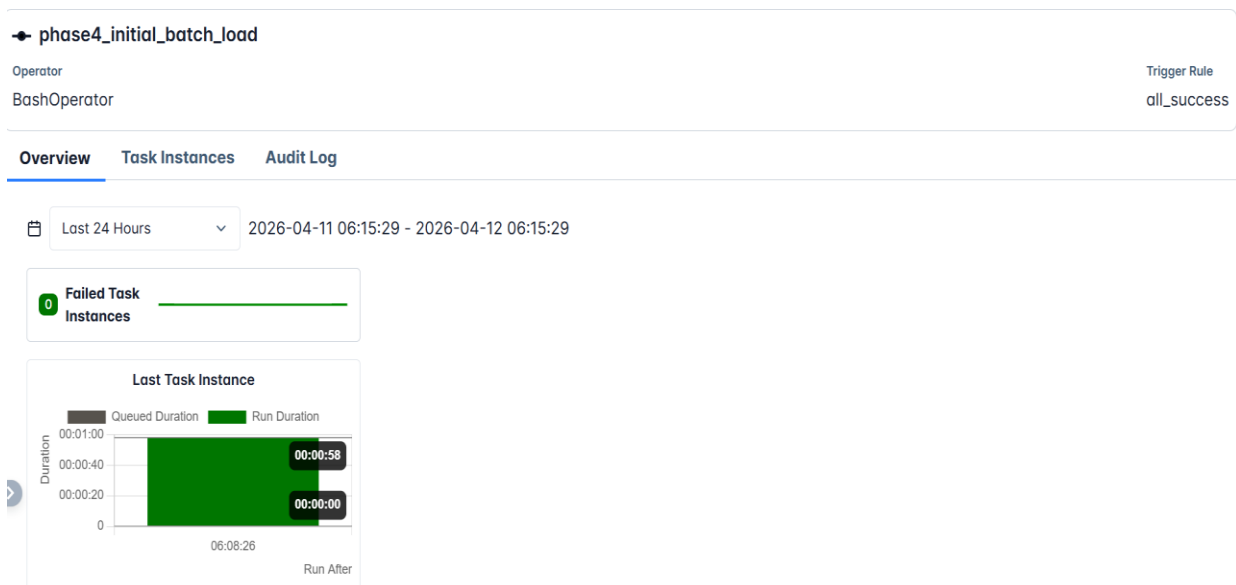


Figure 11: Execution of Phase 4: Initial Batch Ingestion Pipeline

This figure demonstrates batch ingestion of healthcare datasets, including validation, envelope generation, and separation into accepted and quarantine layers.

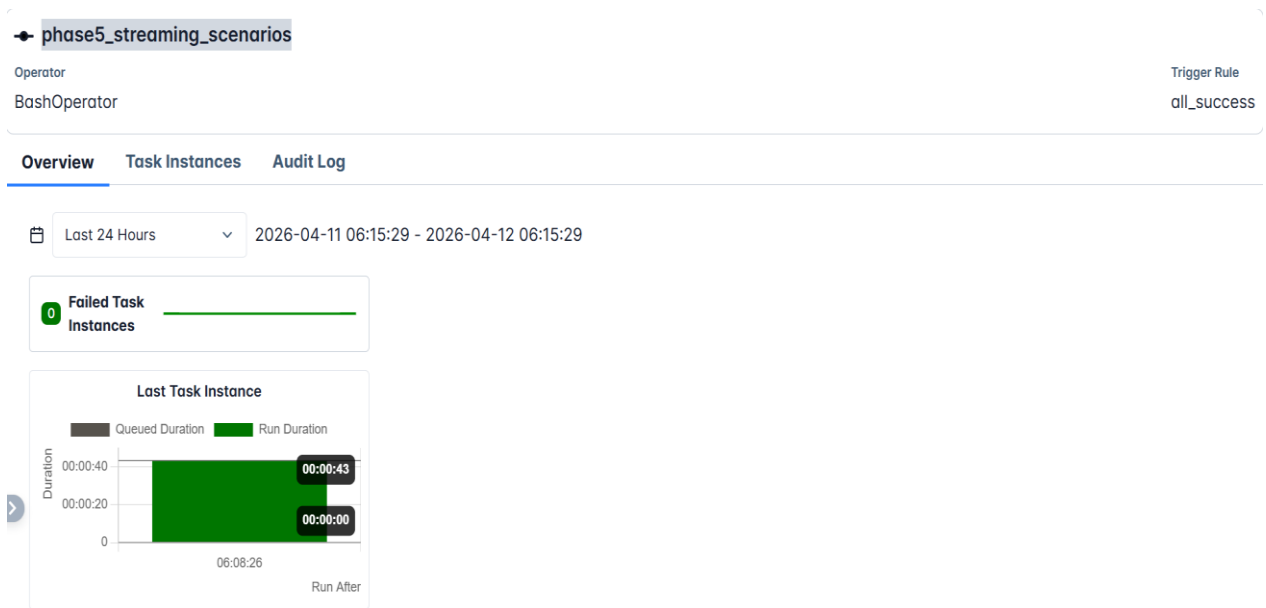


Figure 12: Execution of Phase 5: Streaming Ingestion Scenarios (Steady vs Burst)

This figure illustrates streaming simulation under varying load conditions, enabling evaluation of pipeline behaviour under both continuous and high-throughput burst scenarios.

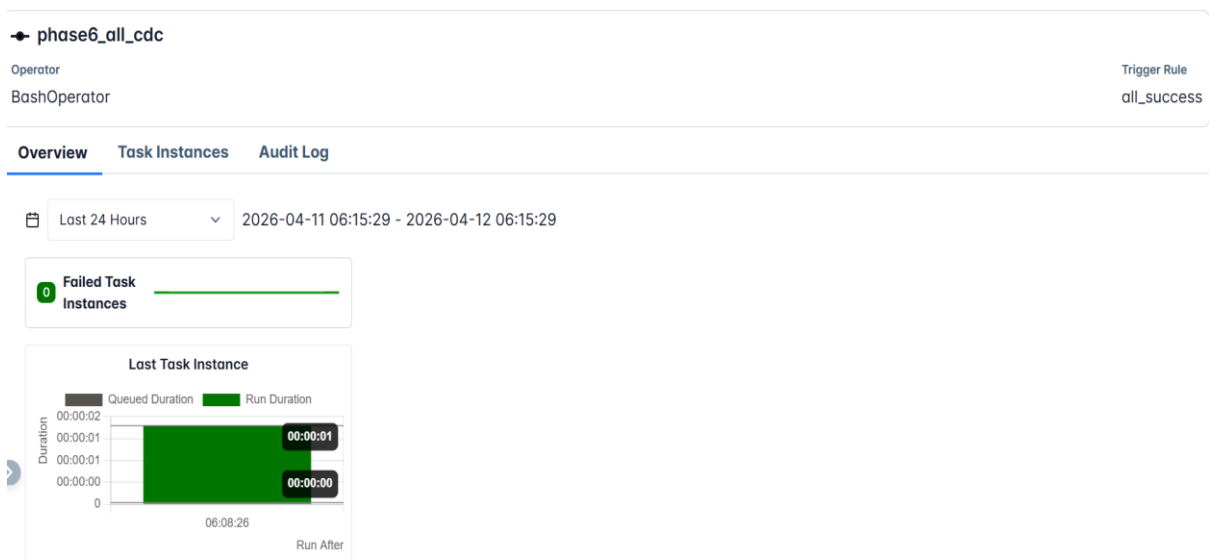


Figure 13: Execution of Phase 6: Change Data Capture Across Multiple Strategies

This figure captures the execution of CDC processes, including timestamp-based, trigger-based, and log-based approaches, demonstrating unified event generation and processing.

phase7\_telemetry\_summary

Success

| Operator     | Start Date          | End Date            | Duration     | Dag Version |
|--------------|---------------------|---------------------|--------------|-------------|
| BashOperator | 2026-04-12 06:10:26 | 2026-04-12 06:10:26 | 00:00:00.700 | v1          |

Logs

Rendered Templates

XCom

Asset Events

Audit Log

Code

Details

All Log Levels

All Sources

```

Log message source details sources=["/opt/airflow/logs/dag_id=de_healthcare_pipeline/run_id>manual__2026-04-12T01:08:26.924527+00:00/task_id=phase7_telemetry_summary/attempt=1.log"]
2 [2026-04-12 06:10:26] INFO - DAG bundles loaded: dags-folder, example_dags
3 [2026-04-12 06:10:26] INFO - Filling up the DagBag from /opt/airflow/dags/de_healthcare_pipeline_dag.py
4 [2026-04-12 06:10:26] INFO - Task instance is in running state
5 [2026-04-12 06:10:26] INFO - Previous state of the Task instance: TaskInstanceState.QUEUED
6 [2026-04-12 06:10:26] INFO - Current task name:phase7_telemetry_summary
7 [2026-04-12 06:10:26] INFO - Dag name:de_healthcare_pipeline
8 [2026-04-12 06:10:26] INFO - Tmp dir root location: /tmp
9 [2026-04-12 06:10:26] INFO - Running command: ['/usr/bin/bash', '-c', 'cd /opt/project && python run_phase7_telemetry_summary.py']
10 [2026-04-12 06:10:26] INFO - Output:
11 [2026-04-12 06:10:26] INFO - Telemetry summary artifacts written successfully.
12 [2026-04-12 06:10:26] INFO - Detail CSV: /opt/project/data/snapshots/telemetry_dashboard_table.csv
13 [2026-04-12 06:10:26] INFO - Detail JSON: /opt/project/data/snapshots/telemetry_dashboard_table.json
14 [2026-04-12 06:10:26] INFO - Summary JSON: /opt/project/data/snapshots/telemetry_dashboard_summary.json
15 [2026-04-12 06:10:26] INFO - Command exited with return code 0
16 [2026-04-12 06:10:26] INFO - Pushing xcom ti=RuntimeTaskInstance(id=UUID('019d7f3b-9ec1-7c80-a9bd-d05e9a0bb8e0'), task_id='phase7_telemetry_summary', dag_id='de_healthcare_pipeline',
17 [2026-04-12 06:10:26] INFO - Task instance in success state
18 [2026-04-12 06:10:26] INFO - Previous state of the Task instance: TaskInstanceState.RUNNING
19 [2026-04-12 06:10:26] INFO - Task operator:<Task(BashOperator): phase7_telemetry_summary>

```

Figure 14: Execution of Phase 7: Telemetry Aggregation and System Summary Generation

This figure shows the final orchestration stage, where telemetry outputs are aggregated into summary artifacts used for observability and dashboard visualization.

The DAG structure demonstrates explicit dependency management, where each task executes only after the successful completion of its predecessor. This aligns with established orchestration principles, where workflow systems act as external coordinators of data processing tasks.

The Airflow interface provides visibility into task execution states, enabling monitoring of success, failure, and retry behaviour. This is critical for operational reliability, particularly in systems where data correctness depends on sequential consistency.

A key engineering insight emerged during implementation: initial pipeline failures due to missing `/opt/project` mounting revealed the importance of environment consistency in containerized systems. Without correct volume mapping, the orchestration layer could not access execution code, effectively breaking the pipeline.

Resolving this required explicit Docker volume configuration, ensuring that Airflow containers had access to the project directory. This debugging process reinforces a fundamental distributed systems principle: execution environments must be carefully aligned across all system components.

## 10. Serving Layer: From Data to Clinical Insight

The serving layer represents the final stage of the data engineering lifecycle, where processed data is transformed into actionable insight. In this system, a Streamlit dashboard is used to present telemetry outputs and CDC performance metrics in an interpretable format.

The dashboard enables:

- Visualization of ingestion throughput and latency
- Comparison of steady versus burst streaming scenarios
- Evaluation of CDC strategies under varying system loads
- Monitoring of data quality through failure metrics

To transition from system-level monitoring to decision-oriented insights, the serving layer presents simplified tabular views of key metrics.

Records Ingested by Job

|   | job_id                     | records_ingested |
|---|----------------------------|------------------|
| 0 | job_initial_load_diagnosis | 18310            |
| 1 | job_initial_load_encounter | 9963             |
| 2 | job_initial_load_labs      | 55962            |
| 3 | job_initial_load_mortality | 187              |
| 4 | job_stream_vitals_burst    | 4979             |
| 5 | job_stream_vitals_steady   | 299              |
| 6 | job_log_cdc                | 4497             |
| 7 | job_log_cdc                | 275              |
| 8 | job_timestamp_cdc_vitals   | 4979             |

Figure 15: Tabular Representation of Records Ingested Across Pipeline Jobs

This table provides a simplified serving-layer view of ingestion volumes, enabling quick comparison across datasets and execution modes for operational monitoring.

Records Failed by Job

|   | job_id                     | records_failed |
|---|----------------------------|----------------|
| 0 | job_initial_load_diagnosis | 686            |
| 1 | job_initial_load_encounter | 37             |
| 2 | job_initial_load_labs      | 1208           |
| 3 | job_initial_load_mortality | 15             |
| 4 | job_stream_vitals_burst    | 21             |
| 5 | job_stream_vitals_steady   | 1              |
| 6 | job_log_cdc                | 21             |
| 7 | job_log_cdc                | 1              |
| 8 | job_timestamp_cdc_vitals   | 21             |

Figure 16: Tabular Representation of Records Failed Across Pipeline Jobs



This view supports rapid identification of failure-prone jobs and datasets, providing actionable insight into data quality issues.

The dashboard serves as a bridge between system internals and human decision-making. Rather than exposing raw logs or JSON outputs, it aggregates and presents key indicators in a format suitable for analysis.

Throughput by Job ↗

|   | job_id                     | throughput_rps |
|---|----------------------------|----------------|
| 0 | job_initial_load_diagnosis | 3382.9358      |
| 1 | job_initial_load_encounter | 3107.7678      |
| 2 | job_initial_load_labs      | 1354.076       |
| 3 | job_initial_load_mortality | 3609.5911      |
| 4 | job_stream_vitals_burst    | 921.4952       |
| 5 | job_stream_vitals_steady   | 8.592          |
| 6 | job_log_cdc                | 974.3425       |
| 7 | job_log_cdc                | 896.9          |
| 8 | job_timestamp_cdc_vitals   | 1364.7348      |

Figure 17: Throughput Comparison Across Jobs in Serving Layer View

This table presents throughput metrics across ingestion strategies, enabling comparison of system performance under batch, streaming, and CDC workloads.

From a healthcare systems perspective, this layer is particularly significant. The pipeline is designed to support clinical use cases such as ICU deterioration monitoring, acute kidney injury detection, and hypoxia tracking. In such contexts, delays, data inconsistencies, or ingestion failures can directly impact downstream analytics and decision support systems.

Thus, the dashboard does not merely visualize system performance; it reflects the reliability of data pipelines that underpin clinical insight generation.

This completes the full data engineering lifecycle:

Data Generation → Ingestion → Transformation → Observability → Orchestration → Serving

The inclusion of a serving layer elevates the system beyond a technical pipeline, demonstrating an end-to-end architecture where data engineering outputs are directly connected to interpretability and decision-making.

## 11. Critical Evaluation

The pipeline demonstrates strong alignment with core data engineering principles, achieving lifecycle completeness from data generation to serving. A key strength is the integration of contract-driven validation with an event-based ingestion model, ensuring data correctness at the point of entry. The explicit separation of accepted and quarantined records, combined with structured telemetry, provides auditability and transparency. Additionally, the unified handling of multiple CDC strategies within a consistent event abstraction reflects sound system design and supports extensibility and correctness.

The primary limitations stem from the use of a single-node environment and simulated components, including CDC mechanisms that approximate rather than directly integrate with database logs. Certain telemetry outputs, such as high latency, are influenced by synthetic timestamp design rather than actual system performance. These are intentional trade-offs, prioritizing conceptual clarity and reproducibility within an academic setting. As a result, the system effectively captures the essential design patterns of production data platforms without unnecessary implementation complexity.

## 12. Conclusion

This project demonstrates a shift from pipeline construction to system-level data engineering design by integrating governance, ingestion, CDC, observability, orchestration, and serving within a coherent architecture. By combining clinical context with theoretical foundations and practical implementation, it models how healthcare data systems must enforce correctness, handle variability, and support reliable downstream use. While simplified for academic purposes, the system captures the key principles required for designing robust, interpretable, and extensible data engineering solutions.

## References

1. Reis, J. and Housley, M. (2022) Fundamentals of Data Engineering: Plan and Build Robust Data Systems. Sebastopol: O'Reilly Media.
2. Kleppmann, M. (2017) Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol: O'Reilly Media.
3. Running Airflow in Docker. Apache Airflow How-to-Guides. Available at: <https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html> (Accessed: 28 March 2026)
4. Annamaneni, R (2025): End-to-End Healthcare ETL Pipeline using Azure Data Factory, Databricks and Delta Lake. Medium. Available at: [medium.com/@rakeshraoannamaneni/end-to-end-healthcare-etl-pipeline-using-azure-data-factory-databricks-and-delta-lake-4ddfc29f65f8](https://medium.com/@rakeshraoannamaneni/end-to-end-healthcare-etl-pipeline-using-azure-data-factory-databricks-and-delta-lake-4ddfc29f65f8) (Accessed 28 March 2026)